Université de Montpellier

Faculté des Sciences

TP 2 — Compréhension et architectures

Construction, exécution et validation

HAI913I — Évolution et restructuration des logiciels

Adrien SAMBARREY

Numéro étudiant : 22510228 — Parcours GL

Pierre PREVOST-AVENEL

Numéro étudiant : 22205821 — Parcours GL

Enseignant responsable : Bachar Rima

Année universitaire 2026–2027

16 octobre 2026

Table des matières

Note de migration vers le dépôt TP1 + TP2 3

HAI913I — TP2 : Compréhension et architectures 3

1. Projet et architecture . . . 3
2. Environnement réellement utilisé . . . 3
3. Construction depuis une copie fraîche . . . 4
4. Entrées attendues . . . 4
5. Exécution et seuils . . . 4
6. Sorties et rechargement . . . 5
7. Validation attendue . . . 6
8. Dépannage et limites . . . 7
9. Bonus : commandes et reproduction . . . 7
   - C1 — Lien simple . . . 7
   - C2 — Relations structurelles . . . 7
   - C3 — Interface Swing . . . 8
   - C4 — Réextraction et second projet . . . 8
10. Rapport, vidéo et usage de l'IA . . . 9

Note de migration vers le dépôt TP1 + TP2

Ce document décrit le rendu TP2 validé le 16 octobre 2026. Le dépôt a ensuite intégré son moteur au projet TP1 le 17 octobre : les sources sont réunies sous src/, le seul JAR devient target/hai913i-tp1-analyzer.jar et la commande tp2 --sources relie directement les faits du TP1 au moteur. Les résultats et la vidéo du rendu initial restent conservés comme preuves de cette version. Le README principal et doc/INTEGRATION-TP2.md donnent les commandes actuelles et distinguent les validations antérieures de la compilation de l'intégration. Les chemins source/ ou bonus/ cités dans le texte historique correspondent à l'ancien paquet autonome.

HAI913I — TP2 : Compréhension et architectures

Adrien SAMBARREY (22510228) et Pierre PREVOST-AVENEL (22205821), parcours GL. Université de Montpellier, Faculté des Sciences — Bachar Rima — 2026–2027. Date de réalisation : 16 octobre 2026.

1. Projet et architecture

Le programme charge des faits d'appel, calcule C(A,B), construit un graphe pondéré, applique le clustering ascendant par lien moyen et coupe le dendrogramme selon CP. Chemin 2 du sujet : faits canoniques fournis. L'extracteur du TP1 n'est pas requis. Les parties A0 à B4 et les quatre bonus C1 à C4 sont traités. L'extracteur TP1 est requis seulement pour rejouer l'extraction C4 ; les faits déjà extraits sont fournis.

Les sources Java sont dans src/main/java/hai913i/tp2. FactsSource isole l'entrée ; CsvFactsSource lit les CSV ; Facts contient les faits validés ; CouplingCalculator calcule les poids ; WeightedGraph conserve nœuds et arêtes ; AgglomerativeClusterer produit les ClusterNode ; ModuleCutter retient les branches ; Exports sérialise ; Main traite la ligne de commande. Les tests sont dans src/test/java/hai913i/tp2/PipelineTest.java.

À la racine du rendu : Compte-Rendu.pdf et README.pdf ; source/ pour le projet ; sorties/ pour les exports et figures ; documents/ pour les sources des PDF ; video/demonstration.mp4 et les captures terminal. Aucun dépôt distant n'est nécessaire.

2. Environnement réellement utilisé

Élément	Version / rôle
Système	Ubuntu 24.04.5 LTS, Linux x86_64
Java	OpenJDK 17.0.20+8-1-24.04-Ubuntu
Maven	Apache Maven 3.9.9
Compilation	release Java 17, UTF-8
Tests	JUnit Jupiter 5.14.4, dépendance de test uniquement
Plugins Maven	compiler 3.14.1 ; surefire 3.5.4 ; jar 3.4.2
Dépendances d'exécution	Aucune ; bibliothèque standard Java
Graphviz optionnel	dot annonce 2.43.0 ; paquet Ubuntu 2.42.2-9ubuntu0.1
Éditeur / IDE	Pas d'IDE : fichiers produits et commandes exécutées via Codex et Bash 5.2.21
Documents optionnels	Python 3.12.3 ; ReportLab 5.0.1 ; Pillow 12.3.0
Vidéo optionnelle	imageio-ffmpeg 0.6.0 et FFmpeg embarqué 7.0.2-static

Java 17 minimum et Maven dans PATH suffisent pour construire, tester et exécuter l'outil. Maven télécharge ses plugins et JUnit au premier lancement : une connexion réseau est nécessaire au premier build. Les JAR de dépendances ne sont pas inclus dans l'archive. Graphviz, Python et les bibliothèques de documentation ne sont pas nécessaires à l'analyse.

3. Construction depuis une copie fraîche

Décompresser l'archive puis ouvrir un terminal dans son dossier source/. Toutes les commandes des sections 3 à 9 supposent ce répertoire de travail.

```
cd SAMBARREY_PREVOST-AVENEL_HAI913I_TP2/source
java -version
mvn -v
mvn -B -Dstyle.color=never clean package
```

Le résultat est target/hai913i-tp2.jar. Les 24 tests doivent passer. L'archive remise exclut target/, les JAR, les caches et les dépendances téléchargées. Pour ne lancer que les tests :

```
mvn clean test
```

Une voie sans Maven permet d'exécuter le programme seul, sans JUnit. Depuis source/, dans Bash :

```
mkdir -p target/classes
javac --release 17 -encoding UTF-8 -d target/classes src/main/java/hai913i/tp2/*.java
jar --create --file target/hai913i-tp2.jar --main-class hai913i.tp2.Main -C target/classes .
```

Cette voie a été vérifiée sur une copie fraîche ; elle ne remplace pas la validation JUnit fournie par Maven.

4. Entrées attendues

Un dossier contient trois fichiers CSV UTF-8 de même préfixe, séparés par des points-virgules et avec les en-têtes ci-dessous. Aucun champ du format ne contient de point-virgule, guillemet ou saut de ligne. Les fins de ligne LF et CRLF sont acceptées.

```
<préfixe>-classes.csv : class;simpleName;package
<préfixe>-methods.csv : method;owner
<préfixe>-calls.csv   : caller;callerClass;callee;calleeClass;sites
```

La liste des classes est explicite, y compris les isolés. Les identifiants de méthodes sont uniques, leurs propriétaires doivent exister. Les méthodes des arcs doivent exister et correspondre aux classes annoncées. Un couple caller/callee ne doit apparaître qu'une fois ; sites est un entier positif, ignoré par la métrique. Les fichiers vides, en-têtes incorrects, doublons et références inconnues sont rejetés. Les noms et identifiants contenant les séparateurs réservés (guillemet, point-virgule, barre verticale ou saut de ligne) sont refusés. Le jeu doit comporter au moins une classe ; il peut ne comporter aucun appel.

resources/ contient le jeu canonique facts-validation et son README original. examples/ contient le micro jeu facts-mini et le README original des exemples. Le programme lit les CSV, pas le JSON. Il découvre un préfixe unique ; en cas d'ambiguïté, utiliser --prefix facts-validation. Un préfixe n'est pas un chemin de fichier.

5. Exécution et seuils

CP est obligatoire, fini et compris entre 0 et 1. Utiliser le point décimal, même avec un système en français. --out désigne le dossier d'exports, créé si nécessaire ; sans cette option, il vaut sorties/ dans le répertoire courant. Préférer un dossier distinct pour chaque exécution ; les fichiers de même nom sont remplacés.

```
java -jar target/hai913i-tp2.jar --help
java -jar target/hai913i-tp2.jar resources --cp 0.025 --out ../../sorties/cp-0.025
java -jar target/hai913i-tp2.jar resources --prefix facts-validation --cp 0.015 --out
    ../../sorties/cp-0.015
java -jar target/hai913i-tp2.jar resources --cp 0.045 --out ../../sorties/cp-0.045
```

La dernière commande renvoie 3 : c'est le rejet attendu de la coupe, et non une panne de l'algorithme. Les exports sont conservés et modules.csv porte accepted=false. Dans un script avec set -e, traiter ce code explicitement, comme le fait scripts/valider.sh.

```
java -jar target/hai913i-tp2.jar examples --cp 0.20 --out ../../sorties/mini
java -jar target/hai913i-tp2.jar examples --cp 0.50 --out ../../sorties/mini-rejet
```

Le micro jeu donne un module à 0,20 et deux à 0,50 ; le plafond est floor(3/2)=1, donc la seconde commande renvoie également 3.

Code	Signification
0	Analyse terminée, coupe acceptée ; ou aide affichée
2	Argument, fichier ou dossier invalide ; erreur d'entrée/sortie
3	Analyse terminée et exports écrits, coupe rejetée car plus de floor(M/2) modules

6. Sorties et rechargement

Fichier	Contenu
parametres.txt	Liaison, relation et seuil utilisés
resultats.txt	Compteurs, couples triés, fusions, modules et statut
graphe.csv	Nœuds explicites et arêtes, poids double préservé
graphe.dot	Graphe de couplage non orienté, isolés inclus
couplages.csv	Matrice complète C, diagonale nulle
fusions.csv	Étape, identifiants, similarité, moyenne interne et membres
dendrogramme.txt	Arbre indenté avec identifiants et similarités
dendrogramme.dot	Arbre Graphviz avec ses deux fils par fusion
modules.csv	Module, branche, taille, moyenne interne ou NA, CP, accepted et classes
modules.dot	Classes groupées par module, arêtes internes et externes

Le résumé d'export est commun à la ligne de commande et à l'interface : modifier CP puis réexporter ne laisse pas un ancien statut dans resultats.txt. Le format graphe.csv a l'en-tête kind;classA;classB;weight. Une ligne node;Nom;; conserve un nœud. Une ligne edge;A;B;0.1 conserve une arête non orientée. Une paire n'apparaît qu'une fois, les deux classes doivent exister et le poids doit être fini dans]0,1]. Un nœud sans arête reste présent. Un graphe non vide doit avoir une somme des poids égale à 1 à 10^{-9} près ; une somme différente est refusée pour protéger l'échelle de CP. Un graphe sans arête est accepté avec une somme nulle. L'ordre des lignes n'influence pas le résultat.

```
java -jar target/hai913i-tp2.jar --graph ../../sorties/cp-0.025/graphe.csv --cp 0.025 --out
    target/recharge
cmp ../../sorties/cp-0.025/modules.csv target/recharge/modules.csv
```

Le mode --graph ne connaît plus les méthodes ou les arcs initiaux et n'affiche pas leurs compteurs. Les couplages, fusions et modules restent identiques. L'export de graphe utilise la précision complète des

doubles, les valeurs console étant arrondies pour la lecture. Dans fusions.csv et modules.csv, les classes d'un groupe sont séparées par le caractère barre verticale.

Les figures déjà fournies dans sorties/figures sont consultables sans installer Graphviz. Pour produire soi-même des images des DOT bruts, avec Graphviz dans PATH :

```
dot -V
sh scripts/images.sh ../../sorties/cp-0.025
```

Le script produit PNG et SVG. Les figures du rapport utilisent des noms courts ; leurs DOT dédiés sont fournis dans sorties/figures. Le dendrogramme par étapes est une figure complémentaire, tandis que le DOT brut conserve l'arbre complet et les noms qualifiés.

7. Validation attendue

```
sh scripts/valider.sh
```

Ce script lance mvn clean verify, les deux coupes du micro jeu, les trois seuils canoniques, un second passage identique et le rechargement du graphe. Il vérifie les codes de sortie attendus puis compare les exports octet par octet. Le message final doit être VALIDATION TERMINÉE. Les diagnostics d'erreur d'entrée et de coupe rejetée font partie de ce protocole et sont attendus.

Contrôle	Résultat attendu
A0 canonique	15 classes, 59 méthodes, 44 arcs
A0 micro jeu	3 classes, 5 méthodes, 4 arcs
A1 canonique	7 arcs internes, 37 interclasses ; somme C = 1
A2 canonique	15 nœuds, 19 arêtes ; Dvd et LoanException isolés
B1 canonique	14 fusions, deux dernières à zéro ; départ 4/37, 3/37, 9/148
B2 canonique	15 feuilles et 14 nœuds internes
B3 CP=0,015	4 modules, tailles 11, 1, 1, 2 ; accepté
B3 CP=0,025	7 modules, tailles 7, 1, 1, 1, 2, 1, 2 ; accepté
B3 CP=0,045	10 modules, maximum 7 ; rejeté, code 3

Les résultats réellement obtenus sont conservés dans ../sorties, avec validation.log. La somme 1 porte sur les paires non ordonnées et suppose au moins un arc interclasse. Sans relation interclasse, tous les poids et leur somme sont nuls ; en variante C2, un héritage interne peut toutefois créer un couplage même en l'absence d'appel. À une seule classe, la règle littérale floor(M/2)=0 rejette le module unique ; ce cas est testé.

8. Dépannage et limites

Symptôme	Remède / explication
mvn introuvable	Installer Maven ou ajouter son dossier bin au PATH ; voie javac possible pour le programme seul
Java trop ancien	Installer et sélectionner un JDK 17 ou supérieur, vérifier java et javac
Téléchargement Maven impossible	Vérifier réseau, proxy et accès à Maven Central ; relancer après correction
Dossier ou fichier absent	Vérifier le répertoire courant et le chemin relatif ; lancer depuis source/
Plusieurs jeux détectés	Utiliser --prefix avec le préfixe commun des trois CSV
CP absent, NaN ou 0,025 refusé	Donner --cp 0.025 ; les valeurs non finies sont invalides
Code 3	La coupe viole M/2 ; examiner les modules et choisir un seuil justifié
Couplages faibles	Normalisation par le nombre global d'arcs interclasses ; CP n'est pas un pourcentage local
Dvd et LoanException isolés	L'héritage, les constructions et les levées d'exceptions ne comptent pas
dot introuvable	Ouvrir les images fournies ou installer Graphviz ; l'outil Java reste utilisable

Le moteur ne lit pas le JSON. Le script optionnel heritage.py convertit son bloc structurel en CSV ; le sous-projet TP1 réalise l'extraction C4. Par défaut, les partitions suivent les appels statiques des faits ; la variante --inheritance ajoute les relations structurelles explicitement fournies. Elles doivent être interprétées avant toute restructuration. L'outil est conçu pour des projets modérés : la version directe du clustering fait $O(M^3)$ lectures de poids dans le pire cas.

9. Bonus : commandes et reproduction

C1 — Lien simple

```
java -jar target/hai913i-tp2.jar resources --cp 0.025 --linkage single --out ../../sorties/c1-lien-
    simple
```

--linkage average est la valeur par défaut. Le lien simple prend le maximum des C(a,b), le lien moyen leur moyenne. Les arbres divergent dès l'étape 2 : (0,7) au lien moyen, (11,15) au lien simple. La coupe garde sa définition par moyenne interne, même lorsque la liaison change. Les deux variantes donnent sept modules à 0,025, avec des compositions différentes.

C2 — Relations structurelles

```
python3 scripts/heritage.py
java -jar target/hai913i-tp2.jar resources --cp 0.025 --inheritance resources/facts-validation-
    inheritance.csv --out ../../sorties/c2-relations-enrichies
java -jar target/hai913i-tp2.jar resources --cp 0.020 --inheritance resources/facts-validation-
    inheritance.csv --out ../../sorties/c2-cp-0.020
```

Le CSV structurel a pour en-tête type;target;kind ; kind vaut extends ou implements. Une relation interne distincte compte une unité, comme un arc interclasse. Les relations vers des types externes sont ignorées avec un compteur. Le dénominateur vaut ici 37+4=41 ; 22 arêtes et un seul isolé. À 0,025, huit modules : code 3 attendu ; à 0,020, sept modules : code 0. L'option exige les faits initiaux ; elle est incompatible avec

--graph, qui ne contient plus les comptes d'arcs. parametres.txt indique les options utilisées.

C3 — Interface Swing

```
java -jar target/hai913i-tp2.jar resources --cp 0.025 --gui --out target/gui
java -jar target/hai913i-tp2.jar --graph ../../sorties/c2-cp-0.020/graphe.csv --cp 0.020 --gui --out
    target/gui-enrichi
```

Un écran graphique est nécessaire ; en session sans écran, le programme renvoie 2 avec un diagnostic. La fenêtre permet de rechercher et sélectionner une classe, d'inspecter ses couplages, de parcourir l'arbre, de changer CP et le critère de liaison, de filtrer les arêtes visibles et d'exporter le résultat courant. Le filtre de recherche et celui des arêtes ne changent pas le calcul. Un changement de liaison efface l'inspection ancienne ; les sélections d'une classe et d'une branche sont mutuellement exclusives. Les couleurs représentent les modules ; la disposition circulaire ne représente pas une distance.

Le terminal et les exports initiaux décrivent les paramètres du lancement. Les changements dans la fenêtre restent en mémoire jusqu'au bouton Exporter : choisir alors un dossier distinct. L'export graphique écrit les paramètres courants dans parametres.txt et régénère resultats.txt par le même code que la CLI. La provenance de la relation est conservée. À la fermeture de la fenêtre lancée par la CLI, le processus renvoie 0 ou 3 selon la coupe courante. Une coupe rejetée reste inspectable et exportable comme diagnostic.

Les tests du modèle de l'interface sont inclus dans les 24 tests. Pour rejouer le test réel de la fenêtre et ses captures, avec DISPLAY ou une session X11 virtuelle :

```
mkdir -p target/smoke
javac --release 17 -encoding UTF-8 -cp target/hai913i-tp2.jar -d target/smoke bonus/GuiSmoke.java
java -cp target/smoke:target/hai913i-tp2.jar hai913i.tp2.GuiSmoke target/gui-captures-x11
```

Sous Windows, remplacer le séparateur de classpath : par ;. Ce test agit sur les composants Swing réels et vérifie les états ; Robot capture la fenêtre. Sous certaines sessions Wayland, une capture peut être noire malgré une fenêtre fonctionnelle : utiliser une session X11 ou Xvfb pour la capture. Le test couvre aussi le véritable dialogue de dossier d'export et vérifie resultats.txt après modification de CP. La vidéo fournie a été capturée sur un écran X11 virtuel, sans image d'interface simulée.

C4 — Réextraction et second projet

Le second système est le code de l'analyseur TP1, fourni dans bonus/tp1-extracteur. Son parseur et ses visiteurs sont inchangés. bonus/ExportTp2.java utilise réellement ces classes et le constructeur de graphe pour produire les CSV canoniques. Le manifeste bonus/tp1-sources.sha256 identifie les 29 fichiers Java analysés.

```
sh scripts/extraire-tp1.sh bonus/tp1-extracteur target/reextraction
java -jar target/hai913i-tp2.jar target/reextraction --cp 0.010 --out target/c4
# Ou utiliser les faits déjà reconstruits et inclus :
java -jar target/hai913i-tp2.jar datasets/analyseur-tp1 --cp 0.010 --out ../../sorties/c4-cp-0.010
java -jar target/hai913i-tp2.jar datasets/analyseur-tp1 --cp 0.025 --out ../../sorties/c4-cp-0.025
```

La réextraction construit le sous-projet Maven TP1 : JDT Core 3.46.0, Spoon 11.5.0, SLF4J 2.0.18, dépendances séparées du moteur TP2. Spoon reste une dépendance du projet original ; l'extraction exécutée est JDT. Le script fournit le JAR et son classpath à JDT pour résoudre les bibliothèques. Aucun JAR ni cache n'est inclus dans le rendu.

Résultats après complétion : 30 classes, 199 méthodes, 267 arcs, dont 77 internes et 190 interclasses ; 52 arêtes de classes, quatre isolés et 29 fusions. À CP=0,010, quinze modules acceptés ; à CP=0,025, vingt-trois modules rejetés. Le pont réintègre 69 cibles implicites réellement résolues par JDT, soit 121 arcs supplémentaires et 144 sites (143 vers des records, un vers un enum). cibles-reintegrees.csv détaille les signatures, propriétaires et sites ; verifier-extraction.py contrôle leur conservation. Le corpus et les visiteurs TP1 restent inchangés. Les méthodes implicites jamais appelées, les corps synthétiques et les constructeurs ne sont pas reconstruits. Le rapport compare les compositions avant/après : un même nombre de modules ne signifie pas une même partition.

Le protocole complet reconstruit les faits, les compare à ceux livrés, et contrôle aussi le pont sur le projet de validation :

```
sh scripts/valider-bonus.sh
```

Python 3 est utilisé pour la comparaison indépendante des CSV. La fenêtre C3 est vérifiée séparément car elle exige un écran : sh scripts/verifier-gui.sh. Ce script vérifie aussi la fermeture avec les codes 0 et 3 et l'erreur 2 sans écran. Le protocole exécute également scripts/verifier-oracle.py : 112 comparaisons avec des fractions exactes, dont 16 systèmes générés et les deux liaisons. Les empreintes du corpus TP1 sont contrôlées. Le journal complet est ../../sorties/bonus-validation.log ; la comparaison calculée est ../../sorties/bonus-comparaison.json. Les sorties des différentes variantes sont dans des dossiers distincts.

10. Rapport, vidéo et usage de l'IA

Le rapport est ../Compte-Rendu.pdf. La vidéo ../video/demonstration.mp4 couvre à elle seule les obligations et les quatre bonus : c’est une capture continue d’un véritable bureau X11 : VS Code ouvre les sources et son terminal intégré exécute un véritable Bash ; feh affiche les images et Swing fournit C3. Le cadrage code/terminal, les cartouches jaunes, les encadrés sur le code et les explications en bandeau inférieur reprennent le style de la vidéo TP1, sans voix. Elle montre quatre passages dans l’éditeur sur les sources numérotées, colorées et surlignées (couplage, lien moyen, départage et coupe), trois erreurs réellement provoquées (CP absent, CP mal formé, dossier inexistant), la régénération des images Graphviz avec repères jaunes sur les isolés et le module principal, l'extraction TP1 en cours, les coupes acceptée et rejetée, le choix de CP et la paire discutable Member–ConsoleNotifier, les interactions C3 et le dialogue d'export. Les commandes et interactions sont pilotées pour la reproductibilité ; aucune sortie n'est préécrite. La saisie au clavier et les déplacements de souris sont injectés par Robot avec une cadence variable et quelques corrections scénarisées. Le curseur est restitué à sa position réelle. Cette présentation reste explicitement automatisée. Les repères temporels sont ../video/chapitres.txt, les commandes et codes de sortie ../video/executions.log, la transcription du pseudo-terminal ../video/terminal.txt et les métadonnées ../video/metadata.json. La capture ../video/interface-bonus.mp4 est un complément facultatif. Aucun lien externe ni compte n'est requis.

../AUDIT.md détaille les problèmes relevés et leur correction. ../Guide-Soutenance.md rassemble les calculs et explications à examiner avant la remise.

OpenAI Codex, annoncé comme fondé sur GPT-6 dans la session, a produit le code, les tests, la documentation et la capture de démonstration. La section IA du rapport précise l'étendue de cette assistance. Les tests cités ont été exécutés. L'examen et l'appropriation par le binôme avant remise restent nécessaires ; aucune compréhension personnelle non démontrée n'est attestée.

La régénération des documents et de la vidéo est facultative. Depuis la racine du rendu, avec Python 3.12 et Graphviz pour les figures :

```
python3 -m venv /tmp/tp2-doc-venv
/tmp/tp2-doc-venv/bin/pip install -r documents/requirements.txt
/tmp/tp2-doc-venv/bin/python documents/generer.py
# java, javac, mvn, dot, Xvfb, openbox, xdotool, feh et script dans PATH ; VS Code via code ou TP2_CODE
     :
/tmp/tp2-doc-venv/bin/python video/enregistrer.py --xvfb
```

La capture vidéo utilise un écran privé Xvfb sous Linux (paquet xvfb) ; --display :N permet alternativement de choisir un écran X11 existant. La production utilise Xvfb 21.1.12, VS Code 1.142.0 (distribution autonome), Openbox 3.6.1, xdotool 3.20160805.1 et feh 3.10.1, ainsi que script (util-linux). TP2_CODE permet de fournir le chemin du binaire natif de VS Code (par exemple /chemin/VSCode-linux-x64/code). Un profil temporaire isolé définit le thème, les raccourcis et le véritable terminal Bash ; aucun compte n’est nécessaire. Ces outils servent uniquement à filmer le véritable bureau et ne sont pas des dépendances du moteur Java. RealTerminalDemo.java pilote les frappes et attend le retour de l’invite Bash pour vérifier le code de sortie ; enregistrer.py ajoute les cartouches, les explications et l’encadré de sélection, puis encode les captures. Les figures sources et les textes Markdown sont fournis ; les PDF peuvent être régénérés à

partir des images déjà présentes, sans Graphviz. Les dépendances Python sont réservées à cette production documentaire.